



TRABAJO PRÁCTICO INTEGRADOR VIRTUALIZACIÓN

ARQUITECTURA Y SISTEMAS OPERATIVOS

TECNICATURA EN PROGRAMACION

Universidad tecnológica Nacional – Facultad Regional San Nicolás

Gonzalez Nahir Evelyn – 43113814 / Rossi Franco Ian - 42559183

Nahirnahir2025@outlook.com / rossifrancoian@gmail.com

ÍNDICE

1. Introducción	3
2. Marco Teórico	4
2.1 Introducción:	4
2.2 ¿Qué es la virtualización?	4
2.3 Tipos de hipervisores	4
2.4 Oracle VirtualBox.....	4
2.5 Linux Mint como sistema operativo invitado.....	5
2.6 XFCE: El Entorno de Escritorio Liviano	5
2.7 Snapshots	6
2.8 Modos de red en VirtualBox	6
2.9 Bash: Fundamento del Lenguaje de Scripting.....	6
2.10 Control de Privilegios: EUID y sudo	6
2.11 Variables, Sustitución de Comandos y Expansión de Parámetros	7
2.12 Redirección de Salida y Registro de Logs (tee)	7
2.13 Copias de Seguridad con GNU tar	8
2.14 Comandos de Monitoreo del Sistema.....	8
2.16 Manejo de Errores: Operador y Cortocircuito Lógico	9
2.17 La Variable Especial \$? y Verificación de Códigos de Salida	9
2.18 Arreglos (Arrays) en Bash	9
2.19 Exclusión de Archivos en tar (--exclude)	9
2.20 Redirección a /dev/null: Supresión de Salida de Error	10
2.21 Edición de Archivos con sed (Stream Editor)	10
2.22 Búsqueda y Ejecución Combinada con find -exec.....	10
2.23 Información de CPU: nproc y lscpu	10
2.24 Listado de Archivos con ls -lh y Pipes ().....	11
3. Caso Práctico	11
3.1 Instalación y configuración del entorno virtualizado.....	11
3.2 Planteamiento del problema	12
3.3 Desarrollo del script	12
3.4 Validación del funcionamiento	13

4. Metodología Utilizada.....	13
5. Resultados Obtenido.....	15
5.1 Primeros resultados	15
5.2 Problemas al crear el backup	15
5.3 Solución al problema de creación del backup	15
5.4 Persistencia de errores.....	15
5.5 Solución Final	16
5.6 Resultados Finales	16
6. Conclusiones	16
7. Bibliografía	18
8. Anexos.....	20
8.1 Capturas de pantalla de la creación de la configuración de la primera máquina virtual	20
8.2 Capturas de pantalla de la Escritura del primer script.....	21
8.3 Capturas de pantalla de la Ejecución del primer script	23
8.4 Capturas de pantalla creación máquina virtual apta para ejecutar nuestra script	25
8.5 Capturas de pantalla ejecución del script final en la nueva máquina virtual	26

1. Introducción

El trabajo práctico que desarrollaremos aborda el tema de la virtualización, una tecnología que permite ejecutar múltiples sistemas operativos de forma simultánea sobre un único equipo físico. Se eligió este tema porque es una herramienta muy útil en el día a día de cualquier persona que trabaja con tecnología, ya que permite al usuario experimentar con distintos entornos sin necesidad de contar con hardware adicional.

Una de las ventajas más importantes que nos motivó para la elección del tema es la posibilidad de usar Linux dentro de una máquina virtual como Oracle VirtualBox, para todo tipo de propósitos como: realizar pruebas, ensayar configuraciones, explorar el sistema operativo sin miedo a romper nada, o simplemente indagar en cómo funciona. Esto nos pareció perfecto para nosotros que estamos aprendiendo, nos permite experimentar libremente en un entorno aislado y seguro y llevar la teoría a la práctica de manera directa.

La elaboración de este trabajo tiene como finalidad entender los conceptos básicos de la virtualización y los tipos de hipervisores, instalar y configurar una máquina virtual en VirtualBox con Linux Mint, y ejecutar un script dentro de ese entorno para demostrar cómo se aplica todo lo aprendido.

2. Marco Teórico

2.1 Introducción:

El siguiente marco teórico analiza, comando por comando y estructura por estructura, el script de mantenimiento de sistema desarrollado en **Bash** (Bourne-again SHell), el intérprete de línea de comandos estándar en distribución GNU/Linux, en este caso, el desarrollo y ejecución del script analizado se llevaron a cabo dentro de una máquina virtual con **Linux Mint** (entorno de escritorio XFCE), alojada en **Oracle VirtualBox**, un hipervisor de tipo 2 (hosted). En esta arquitectura, el sistema operativo guest no accede directamente al hardware físico, sino que lo hace a través de dos capas: el hipervisor VirtualBox y, por debajo de este, el sistema operativo anfitrión (host). El script automatiza tareas habituales de administración: actualización de paquetes, limpieza del sistema, generación de copias de seguridad, registro de logs y reporte de uso de recursos. Cada sección de este documento fundamenta teóricamente los mecanismos empleados, citando la documentación oficial de cada herramienta.

2.2 ¿Qué es la virtualización?

La virtualización es una tecnología que permite ejecutar múltiples sistemas operativos de forma simultánea sobre un único equipo físico. Para lograrlo, se crea un entorno especial llamado **máquina virtual (VM)**, donde el sistema operativo invitado funciona como si estuviera corriendo en hardware real, sin saber que comparte recursos con otros sistemas. Al equipo físico donde corre todo se lo llama **host**, y al sistema que corre dentro de la VM se lo llama **guest**.

2.3 Tipos de hipervisores

El software que hace posible la virtualización se llama **hipervisor**. Existen dos tipos:

- **Tipo 1 (bare-metal):** corre directamente sobre el hardware, sin necesidad de un sistema operativo base. Se usa en servidores y entornos empresariales (ej: VMware ESXi, Hyper-V).
- **Tipo 2 (hosted):** corre sobre un sistema operativo ya instalado. Es el caso de **Oracle VirtualBox**, que requiere que el host tenga Windows, Linux o macOS instalado previamente.

2.4 Oracle VirtualBox

VirtualBox es una aplicación de virtualización multiplataforma que extiende las capacidades del equipo existente para poder correr múltiples sistemas operativos dentro de múltiples

máquinas virtuales al mismo tiempo. Es software libre, ampliamente usado tanto en entornos de aprendizaje como profesionales.

2.5 Linux Mint como sistema operativo invitado

Linux Mint es definido por su proyecto oficial como un sistema operativo para computadoras de escritorio y portátiles, diseñado para funcionar "out of the box" (listo para usar) y provisto de las aplicaciones que la mayoría de los usuarios necesita desde la instalación, sin requerir configuración adicional. Es una distribución de software libre y de código abierto que se apoya, según su propia documentación, "sobre los hombros de gigantes": está basada en Ubuntu y, a través de este, en Debian, heredando sus repositorios de software, soporte de hardware y ciclo de lanzamientos.

Entre las características declaradas oficialmente que justifican su elección para este trabajo se destacan:

Requiere muy poco mantenimiento gracias a actualizaciones centralizadas, snapshots del sistema y un gestor de actualizaciones propio (Update Manager), lo cual resulta directamente relevante para el script de mantenimiento analizado, que automatiza justamente ese tipo de tareas.

Es gratuito y casi en su totalidad de código abierto.

Adopta el principio KISS ("Keep It Simple, Stupid"), priorizando una experiencia intuitiva y de bajo umbral de aprendizaje, lo que la convierte en una distribución recomendada habitualmente para usuarios que migran desde Windows o que se inician en el uso de Linux, como suele ser el caso en contextos educativos.

Linux Mint se distribuye en distintas ediciones, cada una con un entorno de escritorio diferente: Cinnamon (la edición principal, desarrollada específicamente para Mint), MATE (basado en el clásico GNOME 2) y Xfce, descrita oficialmente como un entorno liviano enfocado en eficiencia y bajo uso de recursos, ideal para hardware más antiguo o limitado, que es la edición utilizada en este trabajo.

2.6 XFCE: El Entorno de Escritorio Liviano

El proyecto oficial de Xfce lo define como un entorno de escritorio liviano para sistemas operativos tipo Unix, cuyo objetivo es ser rápido y consumir pocos recursos del sistema, sin dejar de ser visualmente atractivo y fácil de usar. Según su documentación, Xfce encarna la filosofía Unix tradicional de modularidad y reutilización: está compuesto por varios módulos empaquetados de forma independiente (el gestor de ventanas Xfwm, el panel xfce4-panel, el gestor de escritorio xfdesktop, el administrador de archivos Thunar, entre otros) que en conjunto conforman la experiencia completa del escritorio, pero que pueden combinarse o reemplazarse según las necesidades del usuario.

La elección de Xfce como entorno de escritorio para la máquina virtual resulta particularmente adecuada en el contexto de la virtualización de tipo 2 analizada en este trabajo, dado que su bajo consumo de CPU y memoria RAM reduce la carga adicional que ya impone la doble capa de abstracción del hipervisor hosted (guest → hipervisor → host → hardware). En otras palabras, al requerir menos recursos del sistema guest, Xfce deja más recursos disponibles tanto para el sistema operativo anfitrión como para la propia ejecución de tareas dentro de la VM, como las automatizadas por el script de mantenimiento.

2.7 Snapshots

Una funcionalidad clave de VirtualBox son los **snapshots** (instantáneas). Permiten guardar el estado exacto de una VM en un momento determinado. Si algo falla o se rompe algo al experimentar, se puede volver al estado anterior en segundos. Esto es especialmente útil cuando se está aprendiendo y se quiere probar sin miedo a arruinar el sistema.

2.8 Modos de red en VirtualBox

VirtualBox ofrece distintos modos de red para las VMs:

- **NAT:** la VM accede a internet a través del host, pero no es visible desde la red local.
- **Bridge:** la VM se comporta como un equipo más en la red local, con IP propia.
- **Host-only:** la VM solo puede comunicarse con el host, sin acceso a internet.

2.9 Bash: Fundamento del Lenguaje de Scripting

Bash es el shell por defecto del sistema operativo GNU, descrito oficialmente como un intérprete de comandos que funciona simultáneamente como interfaz de usuario y como lenguaje de programación, permitiendo combinar utilidades del sistema mediante archivos de comandos (scripts) que adquieren el mismo estatus que los comandos del sistema. Un script de shell es, formalmente, un archivo de texto que contiene comandos del shell; cuando ese archivo se usa como primer argumento al invocar Bash, este lee y ejecuta los comandos del archivo y luego termina. La línea inicial `#!/bin/bash` ("shebang") indica al sistema operativo qué intérprete debe usarse para ejecutar el resto del archivo, garantizando que se use Bash aunque el script se invoque desde otro shell.

2.10 Control de Privilegios: EUID y sudo

El script inicia verificando privilegios de administrador mediante la condición `if [ "$EUID" -ne 0 ]`. La variable EUID es una variable especial de Bash que contiene el ID de usuario efectivo numérico del proceso actual; el valor 0 corresponde siempre al superusuario (root) en sistemas tipo Unix. Esta comprobación es el mecanismo estándar para que un script exija ejecución con privilegios elevados antes de realizar operaciones críticas como instalar

paquetes. Posteriormente, el script utiliza SUDO_USER, una variable de entorno que sudo establece automáticamente con el nombre del usuario que invocó el comando, distinto del usuario destino (root). Según la documentación oficial del proyecto Sudo, cuando sudo es ejecutado por root y la variable SUDO_USER está definida, la política de sudoers utiliza ese valor para determinar quién es el usuario real que originó la acción. Esto permite que el script identifique al usuario real detrás de la sesión elevada y, mediante eval echo "~\$USUARIO_REAL", resuelva su directorio personal real en lugar del de root.

2.11 Variables, Sustitución de Comandos y Expansión de Parámetros

El script declara variables como INICIO=\$(date +%s) o FECHA=\$(date +"%Y-%m-%d_%H-%M-%S"). Según el manual de referencia de Bash, una variable es un parámetro denotado por un nombre que almacena un valor, y se asigna mediante una sentencia de la forma nombre=valor. La sintaxis \$(comando) corresponde a la sustitución de comandos, que permite capturar la salida estándar de un comando y asignarla a una variable, en este caso el resultado del comando date con distintos formatos de salida.

También se observa el uso de la expansión \${SUDO_USER:-\$USER}, una forma de sustitución de parámetros propia de Bash que devuelve el valor de SUDO_USER si está definido y no es nulo, o el valor de USER en caso contrario. Esta construcción es estándar en scripts que deben funcionar tanto si se ejecutan con sudo como si se ejecutan ya como root directamente.

2.12 Redirección de Salida y Registro de Logs (tee)

El núcleo del mantenimiento se apoya en APT (Advanced Package Tool), el sistema de gestión de paquetes de Debian y derivados como Ubuntu. Según la documentación oficial de Debian y el manual de apt-get, cada subcomando cumple una función específica:

Comando	Función según documentación oficial
apt update	Resincroniza los índices de paquetes desde las fuentes definidas en sources.list.
apt upgrade -y	Actualiza los paquetes instalados a sus versiones más recientes disponibles; -y responde automáticamente "sí" a las confirmaciones
apt autoremove -y	Elimina paquetes que fueron instalados automáticamente como dependencias y ya no son requeridos por ningún paquete instalado
Apt autoclean	Limpia la caché de paquetes de tu sistema. A diferencia de una limpieza total, elimina de forma inteligente solo los archivos

	antiguos y obsoletos que ya no se pueden descargar de internet, conservando únicamente los paquetes más recientes que podrías necesitar volver a instalar.
--	--

2.13 Copias de Seguridad con GNU tar

Para el backup, el script ejecuta `tar -czf "$BACKUP_DIR/documentos_$(date +%Y%m%d).tar.gz" "$HOME_USUARIO/Documentos"`. GNU tar es definido en su manual oficial como un programa de archivado diseñado para almacenar el contenido de múltiples archivos dentro de un único archivo (denominado "archivo" o archive), conservando nombres, propietarios y permisos. Las opciones combinadas en el script corresponden a:

- `-c (--create)`: Crea un nuevo archivo tar.
- `-z (--gzip)`: Filtra el archivo a través de gzip para comprimirlo.
- `-f (--file)`: Especifica el nombre del archivo de salida, en este caso el `.tar.gz` con marca de fecha.

2.14 Comandos de Monitoreo del Sistema

El script reporta el estado del sistema utilizando tres utilidades estándar de GNU/Linux, documentadas tanto en el proyecto GNU coreutils como en procps-ng:

Comando	Paquete / Proyecto	Funcion
<code>df -h</code>	GNU coreutils	Reporta el espacio en disco utilizado y disponible por sistema de archivos, en formato legible (-h).
<code>free -h</code>	procps-ng	Muestra la cantidad total, usada y libre de memoria física y de intercambio (swap) del sistema.
<code>uptime</code>	procps-ng	Indica la hora actual, el tiempo que el sistema lleva encendido, la cantidad de usuarios conectados y el promedio de carga
<code>du -sh</code>	GNU coreutils	Calcula el tamaño total (-s, summarize) en formato legible (-h) de un directorio, usado para medir el peso de los backups

Es relevante destacar, según la documentación oficial de procps-ng, que el load average reportado por uptime representa el número promedio de procesos en estado ejecutable o

en espera ininterrumpida de E/S durante los últimos 1, 5 y 15 minutos, una métrica clave para diagnosticar la carga real del servidor tras el mantenimiento.

2.16 Manejo de Errores: Operador y Cortocircuito Lógico

El nuevo bloque que envuelve `apt upgrade` -y incorpora el operador `||` (OR lógico) de Bash:

Según el manual de referencia de GNU Bash, las listas de comandos unidas por `||` se evalúan de izquierda a derecha con **evaluación de cortocircuito** (short-circuit evaluation): el comando del lado derecho solo se ejecuta si el comando del lado izquierdo retorna un código de salida distinto de cero (fallo). Esto le da al script una capacidad de **recuperación ante errores** (error handling) que el script original no tenía: si la primera actualización falla por falta de espacio en disco, automáticamente intenta liberar espacio y reintenta, en lugar de continuar ciegamente o detenerse.

2.17 La Variable Especial `$?` y Verificación de Códigos de Salida

El script ahora verifica explícitamente el resultado de cada backup con `if [ $? -eq 0 ]`. Según el manual de Bash, `$?` es un parámetro especial que se expande al **estado de salida** (exit status) del comando ejecutado más recientemente en primer plano; por convención Unix, **0 indica éxito** y cualquier valor distinto de 0 indica algún tipo de fallo. Esta es la forma estándar y portable de comprobar si un comando se ejecutó correctamente antes de continuar, y es exactamente la mejora que corrige la falla de la versión anterior del script, que asumía éxito en el backup sin verificarlo.

2.18 Arreglos (Arrays) en Bash

La declaración `DIRS_BACKUP=( "$HOME_USUARIO/Documentos" "$HOME_USUARIO/Desktop" . . . )` introduce el uso de **arreglos indexados**, una estructura de datos soportada nativamente por Bash desde la versión 2.0. El manual de referencia los define como variables que pueden contener múltiples valores simultáneamente, accesibles mediante un índice numérico. La expansión `"${DIRS_BACKUP[@]}"` dentro del bucle `for` recorre cada elemento del arreglo como una palabra independiente (gracias a las comillas dobles, que preservan espacios en nombres de carpeta), lo que permite iterar y respaldar múltiples directorios sin repetir código, una mejora estructural directa sobre el script original, que solo respaldaba una carpeta de forma fija.

2.19 Exclusión de Archivos en `tar` (`--exclude`)

La declaración `DIRS_BACKUP=( "$HOME_USUARIO/Documentos" "$HOME_USUARIO/Desktop" . . . )` introduce el uso de **arreglos indexados**, una estructura de datos soportada nativamente por Bash desde la versión 2.0. El manual de r

referencia los define como variables que pueden contener múltiples valores simultáneamente, accesibles mediante un índice numérico. La expansión "\${DIRS_BACKUP[@]}" dentro del bucle `for` recorre cada elemento del arreglo como una palabra independiente (gracias a las comillas dobles, que preservan espacios en nombres de carpeta), lo que permite iterar y respaldar múltiples directorios sin repetir código, una mejora estructural directa sobre el script original, que solo respaldaba una carpeta de forma fija.

2.20 Redirección a /dev/null: Supresión de Salida de Error

El fragmento `"$DIR" 2>/dev/null` redirige el descriptor de error estándar (2) hacia `/dev/null`, un archivo especial documentado en la mayoría de los sistemas Unix/Linux como un "dispositivo nulo" (null device) que descarta silenciosamente todo lo que se escribe en él. Esto se usa típicamente para suprimir mensajes de advertencia no críticos de `tar` (por ejemplo, archivos que cambiaron mientras se leían) sin ocultar el código de salida real del comando, que sigue siendo verificado correctamente con `$?` en la línea siguiente.

2.21 Edición de Archivos con sed (Stream Editor)

La línea `sed -i '/cdrom:/d' /etc/apt/sources.list` utiliza **sed**, descrito en su manual de GNU como un editor de flujo (*stream editor*) no interactivo que aplica transformaciones de texto a un flujo de entrada. El comando `/cdrom:/d` combina una dirección por patrón (`/cdrom:/`, que selecciona las líneas que contienen ese texto) con el comando `d` (delete), eliminando esas líneas. La opción `-i` indica edición *in-place*, es decir, que `sed` modifica el archivo original directamente en lugar de imprimir el resultado por la salida estándar. Esta línea elimina del listado de repositorios cualquier fuente apuntando a un CD-ROM, una causa común de fallos en `apt update` en instalaciones realizadas desde medio físico o ISO.

2.22 Búsqueda y Ejecución Combinada con find -exec

La línea `find /etc/apt/sources.list.d/ -type f -exec sed -i '/cdrom:/d' {} \;` combina dos herramientas. Según el manual de GNU `findutils`, **find** recorre recursivamente una jerarquía de directorios localizando archivos que cumplen ciertos criterios (`-type f` selecciona solo archivos regulares), y la opción `-exec` comando `{}` `\;` ejecuta el comando especificado una vez por cada archivo encontrado, sustituyendo `{}` por la ruta de ese archivo. En este caso, aplica la misma limpieza de `sed` a todos los archivos de configuración adicionales de APT dentro de `sources.list.d/`, no solo al archivo principal.

2.23 Información de CPU: nproc y lscpu

El script agrega `nproc` y `lscpu | grep "Model name"`. Según su documentación (parte de GNU `coreutils`), **nproc** imprime el número de unidades de procesamiento disponibles

para el proceso actual, útil para diagnosticar cuántos núcleos virtuales tiene asignada la VM. **lscpu**, parte del paquete *util-linux*, recopila información de la arquitectura de CPU desde `sysfs`, `/proc/cpuinfo` y otras fuentes del kernel, incluyendo el campo "Model name" que identifica el modelo del procesador (en este caso, el procesador *virtual* que VirtualBox expone al sistema guest).

2.24 Listado de Archivos con `ls -lh` y Pipes (`|`)

La línea `ls -lh "$BACKUP_DIR" | tail -5` introduce el uso de una **tubería (pipe)**, mecanismo fundamental de Unix descrito en el manual de Bash como un operador que conecta la salida estándar de un comando con la entrada estándar del siguiente. Aquí, `ls -lh` (listado largo, formato legible) genera el listado completo de backups, y `tail -5` —de GNU coreutils— filtra y muestra solamente las últimas 5 líneas, mostrando así los backups más recientes sin saturar la salida del log.

3. Caso Práctico

3.1 Instalación y configuración del entorno virtualizado

Siguiendo los pasos indicados por la cátedra en el material audiovisual de referencia, se descargó e instaló en el equipo host la versión de Virtual Box para Windows más actualizada **VirtualBox 7.2.10**, obtenida directamente desde el sitio oficial [virtualbox.org](https://www.virtualbox.org) junto con su paquete de extensión correspondiente.

Una vez instalado, se creó una nueva máquina virtual desde el asistente de creación, configurando los parámetros básicos: nombre de la VM, imagen ISO (Linux) y versión (Linux Mint de 64 bits).

La interfaz también nos permite ingresar un usuario y contraseña, paso que no debemos olvidar ya que, si no, no tendremos forma de acceder a estos datos en un futuro.

A la máquina virtual se le asignaron los siguientes recursos de hardware:

- **Memoria RAM:** 2 GB
- **Procesador:** 2 núcleos
- **Disco duro virtual:** 20 GB

Como sistema operativo invitado se descargó desde su sitio oficial linuxmint.com la imagen ISO de **Linux Mint, edición XFCE**, elegimos esta variante ya que está diseñada para consumir poca RAM y CPU, esto lo hace muy ligero debido a su bajo consumo de recursos y nos resultó más adecuado para un entorno virtualizado con recursos limitados como nuestra máquina virtual. La ISO se montó en la unidad óptica virtual de la VM y se inició el proceso de instalación, siguiendo el asistente gráfico hasta completar la configuración de usuario,

idioma y particionado de disco. Finalizada la instalación, se reinició la máquina virtual y se verificó el correcto arranque del sistema dentro del entorno virtualizado.

Como se detalla en la sección de Resultados Obtenidos, esta configuración inicial resultó insuficiente para ejecutar correctamente el script de mantenimiento desarrollado, por lo que posteriormente se ajustaron los recursos de la máquina virtual a **4 GB de RAM y 30 GB de disco**, configuración con la cual el entorno funcionó de manera estable.

3.2 Planteamiento del problema

Una vez disponible el entorno funcional, se planteó como caso práctico la necesidad de automatizar las tareas habituales de mantenimiento de un sistema operativo Linux. Normalmente, estas tareas requieren la ejecución manual y por separado de varios comandos: actualizar los paquetes del sistema, eliminar paquetes obsoletos, limpiar archivos de caché, generar copias de seguridad de información importante y revisar el estado de los recursos del equipo (disco y memoria). Realizar esto manualmente de forma recurrente resulta tedioso y propenso a omisiones.

3.3 Desarrollo del script

Para resolver este problema se diseñó un script en Bash, ejecutado desde la terminal de Linux Mint con privilegios de superusuario. El script se organiza en bloques claramente diferenciados:

- Comprobación de permisos: el script verifica al inicio que se esté ejecutando con privilegios de root (mediante sudo); si no es así, finaliza mostrando un mensaje de aviso.
- Definición de variables: se establecen variables con la fecha y hora actual, el usuario real que invocó el script (incluso usando sudo), y la ubicación donde se guardarán los backups y el archivo de log.
- Redirección de salida: mediante el comando tee, toda la salida que se muestra en pantalla se vuelca simultáneamente a un archivo de log, permitiendo conservar un registro del proceso.
- Actualización del sistema: se ejecutan apt update y apt upgrade -y para actualizar la lista de paquetes disponibles y los programas instalados.
- Limpieza: se utilizan apt autoremove -y y apt autoclean para eliminar paquetes que ya no son necesarios y limpiar la caché de descargas.
- Backup: se genera una copia comprimida (tar.gz) de la carpeta Documentos del usuario.
- Información del sistema: se muestran el uso del disco (df -h), la memoria RAM disponible (free -h) y el tiempo de actividad del equipo (uptime).

- Cálculo del tiempo de ejecución: mediante una resta de tiempo final de ejecución - tiempo inicial de ejecución, el script informa cuánto tiempo demandó todo el proceso de mantenimiento.

Durante las primeras ejecuciones del script en la máquina virtual, surgieron dos inconvenientes puntuales (detallados en la sección de Resultados Obtenidos): la actualización de paquetes no pudo completarse por falta de espacio en disco, y la creación del backup falló porque la carpeta Documentos no existía en el sistema instalado. A partir de esto, se realizaron dos modificaciones al script original:

- Se incorporó un bloque de **limpieza previa** (`apt clean` y `apt autoclean -y`) antes de iniciar la actualización, con el objetivo de liberar espacio en el caché de paquetes.
- Se modificó el bloque de backup agregando una **validación condicional** (`if [ -d "$CARPETA_DOCS" ]`), de forma que, si la carpeta Documentos no existe, el script genera en su lugar una copia de seguridad del directorio home completo del usuario, evitando así que el proceso falle por la ausencia de una carpeta específica.

3.4 Validación del funcionamiento

Con el script corregido, se volvió a probar su funcionamiento dentro de la máquina virtual. Si bien las mejoras al código solucionaron el problema puntual del backup, los errores de espacio en la actualización persistieron, lo que llevó a identificar que la causa de fondo no estaba en el script sino en los recursos de hardware asignados a la VM. Tras aumentar la configuración a 4 GB de RAM y 30 GB de disco, se ejecutó nuevamente el script, completándose esta vez sin errores: la actualización de paquetes finalizó correctamente, el backup se generó con éxito, y se mostró la información completa del sistema junto con el archivo de log correspondiente. La ejecución final del script, así como el proceso completo de instalación y configuración del entorno virtualizado, se presentan en detalle en el repositorio de GitHub creado para este trabajo práctico.

4. Metodología Utilizada

Como primera etapa, descargamos Oracle VirtualBox y la imagen ISO de Linux Mint XFCE, herramientas necesarias para crear el entorno de trabajo virtualizado.

Posteriormente, creamos un repositorio colaborativo en GitHub, el cual utilizamos para almacenar el código fuente, registrar las modificaciones realizadas y facilitar el trabajo en equipo.

A continuación, instalamos Oracle VirtualBox y configuramos una máquina virtual con Linux Mint XFCE, donde se desarrolló y probó el script de automatización.

Desde el inicio optamos por una metodología de trabajo colaborativa, realizando el análisis, el diseño y el desarrollo del proyecto en conjunto. Consideramos que esta forma de trabajo favorecía el intercambio de conocimientos y permitía que todos los integrantes participaran en cada una de las etapas del proyecto.

Una vez preparado el entorno, investigamos sobre los procesos de automatización mediante Bash, repasando los contenidos de la materia y consultando documentación complementaria. A partir de esta investigación comenzamos la elaboración del marco teórico.

Posteriormente debatimos distintas alternativas de automatización y decidimos desarrollar un script de mantenimiento completo para sistemas Linux, definiendo las funcionalidades que debía incorporar y planificando su estructura.

Con el diseño definido, iniciamos la programación del script Bash dentro de la máquina virtual, implementando gradualmente cada una de sus funciones y realizando pruebas de funcionamiento luego de cada modificación.

Durante el desarrollo surgieron diversas dificultades, tanto relacionadas con la escritura del código como con el rendimiento de la máquina virtual. En varios intentos el script no se ejecutaba correctamente o presentaba problemas de desempeño, por lo que fue necesario revisar tanto la lógica del programa como la configuración del entorno de virtualización.

Luego de analizar las causas, concluimos que una parte importante de los inconvenientes se debía a la configuración de recursos asignados a la máquina virtual. Por este motivo creamos una nueva máquina virtual con mayor cantidad de memoria RAM y espacio de almacenamiento, lo que permitió ejecutar el sistema operativo y el script de forma mucho más eficiente.

Una vez verificado el correcto funcionamiento del script, registramos la versión final en el repositorio de GitHub y documentamos el proceso mediante capturas de pantalla y videos, incluyendo tanto la configuración inicial como la configuración definitiva de la máquina virtual y la ejecución del script.

Finalmente, elaboramos el guion de la presentación y realizamos el video explicativo, donde se muestran las principales características de la virtualización utilizada, el funcionamiento del script y los resultados obtenidos.

5. Resultados Obtenido

5.1 Primeros resultados

Al ejecutar el script con la configuración inicial de la máquina virtual (2 GB de RAM, 2 núcleos y 20 GB de disco), se presentaron varias dificultades. El proceso de actualización detectó correctamente los paquetes para actualizar y los paquetes nuevos para instalar, pero no pudo completarse debido a que no había suficiente espacio disponible en `/var/cache/apt/archives`, devolviendo el error "E: You don't have enough free space in `/var/cache/apt/archives/`". La eliminación de paquetes innecesarios (`apt autoremove`) y la limpieza de caché (`apt autoclean`) se ejecutaron sin inconvenientes, aunque no hubo paquetes que remover en ese momento.

5.2 Problemas al crear el backup

El mayor problema que se nos presentó fue que el script no pudo generar el backup porque la carpeta Documentos no existía en el sistema, devolviendo el error "Cannot stat: No such file or directory". La información del sistema, en cambio, se mostró correctamente: el uso de disco indicó que la partición principal tenía un 28% de uso, la memoria RAM reportó un total de 1.9 GiB con 982 MiB en uso, y el tiempo de actividad se registró sin problemas. A pesar de los errores en la actualización y el backup, el script finalizó indicando "Mantenimiento finalizado correctamente" en 1 segundo, y generó igualmente el archivo de log correspondiente.

5.3 Solución al problema de creación del backup

Para solucionar el problema del backup, se modificó el script agregando una validación con `if [ -d "$CARPETA_DOCS" ]`, de forma que, si la carpeta Documentos no existe, el script genera en su lugar un backup del directorio home completo del usuario. Además, se incorporó un bloque de limpieza previa (`apt clean`, eliminación de archivos en caché y `apt autoclean -y`) para liberar espacio en disco antes de iniciar la actualización de paquetes.

5.4 Persistencia de errores

Al volver a ejecutar el script corregido, los errores persistieron. Esto nos llevó a investigar más a fondo y a identificar que la causa real no estaba en el código, sino en los recursos de hardware asignados a la máquina virtual: tanto la memoria RAM como el espacio en disco resultaban insuficientes para que el sistema operativo funcionara con normalidad y pudiera completar tareas como la descarga de paquetes o la persistencia de archivos y carpetas.

5.5 Solución Final

Para solucionar este problema de falta de recursos de hardware en nuestra máquina virtual, lo que hicimos fue crear una nueva máquina virtual, asignándole en su configuración de hardware nuevos parámetros (**4 GB de RAM y 30 GB de disco**), logrando que nuestra máquina virtual se adapte más a las necesidades de ejecución de nuestro script.

Desde la interfaz de Linux ingresamos al GitHub para cargar nuestro script actualizado.

5.6 Resultados Finales

Por último, ejecutamos nuevamente el script y verificamos que se ejecutó correctamente de principio a fin: la actualización de paquetes se completó sin errores, el backup se generó con éxito, y se mostró correctamente toda la información del sistema junto con el log correspondiente, confirmando el funcionamiento integral del script de mantenimiento.

Esta experiencia permitió comprobar en la práctica un concepto central de la virtualización: la importancia de dimensionar adecuadamente los recursos de hardware virtual asignados a una VM, ya que una configuración insuficiente puede generar fallos que aparentan ser errores de software cuando en realidad son limitaciones de los recursos disponibles.

6. Conclusiones

La realización de este trabajo permitió comprender de manera práctica los conceptos teóricos vistos sobre virtualización, yendo más allá de la simple lectura de definiciones. Instalar y configurar una máquina virtual con VirtualBox, y luego desarrollar y depurar un script real dentro de ese entorno, ayudó a entender que una VM no es solo un "espacio aislado" abstracto, sino un sistema con recursos de hardware concretos (RAM, disco, procesador) que deben dimensionarse correctamente según la tarea que se quiera realizar.

Uno de los principales aprendizajes surgió justamente al enfrentar los errores durante la ejecución del script. En un primer momento, los fallos en la actualización de paquetes y en la creación del backup parecían deberse a problemas en el código. Esto llevó a revisar y corregir la lógica del script (agregando validaciones y una limpieza previa de caché), pero los errores persistieron. Recién al aumentar los recursos asignados a la máquina virtual (de 2 GB de RAM y 20 GB de disco a 4 GB de RAM y 30 GB de disco) el script funcionó correctamente de principio a fin. Esto fue un buen ejercicio de diagnóstico: aprender a distinguir entre un error de software y una limitación de infraestructura es una habilidad fundamental para cualquier técnico en programación, ya que no todos los problemas se resuelven modificando código.

En cuanto a posibles mejoras o extensiones futuras, una de las más relevantes sería incorporar al script un manejo de errores más robusto y didáctico. Actualmente, cuando algo falla durante la ejecución (por ejemplo, al actualizar paquetes o al crear el backup), el script se limita a mostrar la salida cruda de herramientas como apt, dpkg o tar, lo cual puede resultar confuso para un usuario sin conocimientos técnicos avanzados. Sería valioso capturar estos errores de forma específica e interpretarlos, mostrando mensajes claros que indiquen no solo qué falló, sino también una posible causa probable y una sugerencia concreta de solución (por ejemplo, distinguir si un fallo en la actualización se debe a falta de espacio en disco, a problemas de conexión, y orientar al usuario sobre qué hacer en cada caso).

Además de esto, se podría incorporar al script una verificación previa del espacio disponible en disco antes de iniciar la actualización de paquetes, de modo que el proceso se detenga con un mensaje claro y comprensible en lugar de fallar a mitad de camino con un error técnico difícil de interpretar. También sería conveniente ampliar las validaciones para contemplar otras carpetas o archivos importantes del usuario más allá de Documentos, garantizando que el backup cubra la información relevante sin importar cómo esté organizado el sistema de cada usuario.

Por último, el trabajo en equipo resultó clave para resolver las dificultades que fueron apareciendo: el ida y vuelta entre la prueba del script, el análisis de los mensajes de error en pantalla y la propuesta de soluciones permitió avanzar de forma ordenada hasta lograr que el mantenimiento se ejecutara sin errores. En definitiva, el desarrollo de este trabajo práctico permitió no solo afianzar conceptos de arquitectura y sistemas operativos, sino también desarrollar habilidades de resolución de problemas aplicables a cualquier proyecto de programación.

7. Bibliografía

Oracle. (2024). *VirtualBox User Guide for Release 7.2*.

<https://docs.oracle.com/en/virtualization/virtualbox/7.2/user/Introduction.html>

Oracle VirtualBox. (s.f.). *Virtualization*. <https://www.virtualbox.org/wiki/Virtualization>

Linux Mint. (s.f.). *Documentación oficial*. <https://www.linuxmint.com/documentation.php>

Linux Mint. (s.f.). *User Guide*. <https://linuxmint-user-guide.readthedocs.io/>

Linux Mint Team. (s.f.). Home - Linux Mint. <https://www.linuxmint.com/>

Linux Mint Team. (s.f.). Linux Mint - The Linux Mint Community Wiki.

https://mintguide.miraeze.org/wiki/Linux_Mint

Xfce Development Team. (s.f.). Xfce Desktop Environment. <https://xfce.org/>

Xfce Development Team. (s.f.). About – Xfce. <https://xfce.org/about>

GNU Bash Reference Manual — <https://www.gnu.org/software/bash/manual/>

Shell Scripts (Bash Reference Manual) —

https://www.gnu.org/software/bash/manual/html_node/Shell-Scripts.html

Sudoers Manual — <https://www.sudo.ws/docs/man/1.8.31/sudoers.man/>

Sudo Manual — <https://www.sudo.ws/docs/man/1.8.22/sudo.man/>

GNU Bash Reference Manual — <https://www.gnu.org/software/bash/manual/>

apt-get(8) — Debian Manpages — <https://manpages.debian.org/unstable/apt/apt-get.8.en.html>

Debian Handbook — Maintenance and Updates — <https://debian-handbook.info/browse/stable/sect.apt-get.html>

Debian FAQ — Keeping your system up-to-date —

<https://www.debian.org/doc/manuals/debian-faq/uptodate.en.html>

GNU tar Manual — <https://www.gnu.org/software/tar/manual/>

tar(1) — Linux manual page — <https://man7.org/linux/man-pages/man1/tar.1.html>

GNU Bash Reference Manual — Lists of Commands —

https://www.gnu.org/software/bash/manual/html_node/Lists.html

GNU Bash Reference Manual — Special Parameters —

https://www.gnu.org/software/bash/manual/html_node/Special-Parameters.html

GNU Bash Reference Manual — Arrays —

https://www.gnu.org/software/bash/manual/html_node/Arrays.html

GNU tar Manual — Choosing Files and Names —

https://www.gnu.org/software/tar/manual/html_node/exclude.html

GNU Bash Reference Manual — Redirections —

https://www.gnu.org/software/bash/manual/html_node/Redirections.html

GNU sed Manual — <https://www.gnu.org/software/sed/manual/sed.html>

GNU Findutils Manual — Actions —

https://www.gnu.org/software/findutils/manual/html_node/find_html/Actions.html

lscpu(1) — Linux manual page — <https://man7.org/linux/man-pages/man1/lscpu.1.html>

nproc(1) — Linux manual page — <https://man7.org/linux/man-pages/man1/nproc.1.html>

GNU Bash Reference Manual — Pipelines —

https://www.gnu.org/software/bash/manual/html_node/Pipelines.html

tail(1) — Linux manual page — <https://man7.org/linux/man-pages/man1/tail.1.html>

VIRTUALIZACION - <https://www.virtualbox.org/wiki/Virtualization>)

Tipos de virtualizacion -

<https://docs.oracle.com/en/virtualization/virtualbox/7.2/user/Introduction.html>)

Oracle VirtualBox: <https://www.virtualbox.org/manual/topics/Introduction.html>

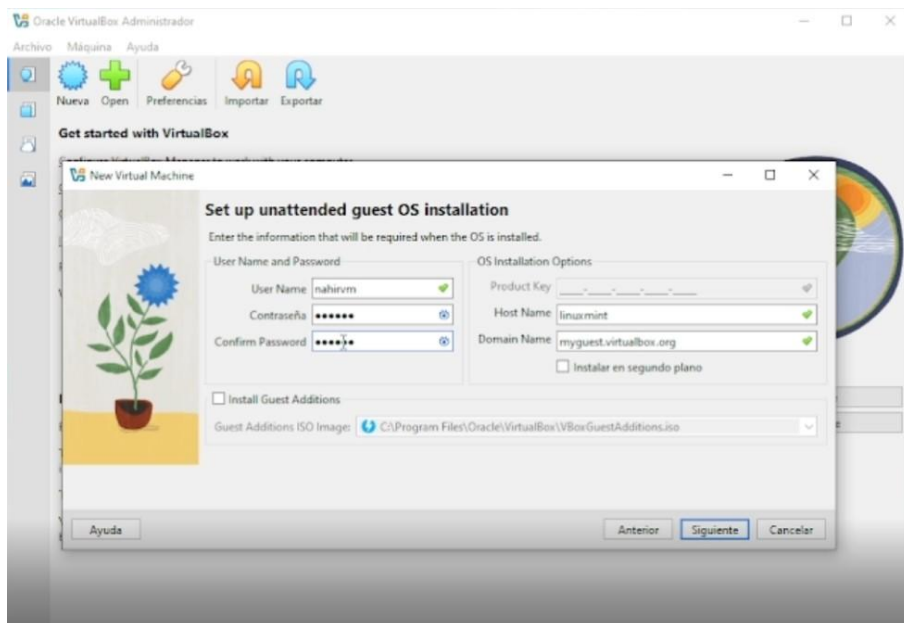
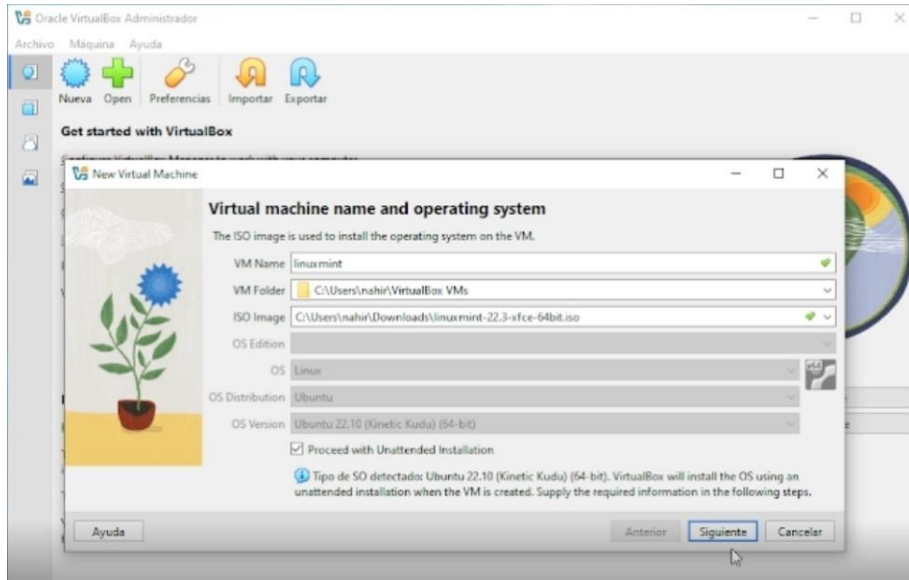
Snapshots: <https://www.virtualbox.org/wiki/Virtualization>

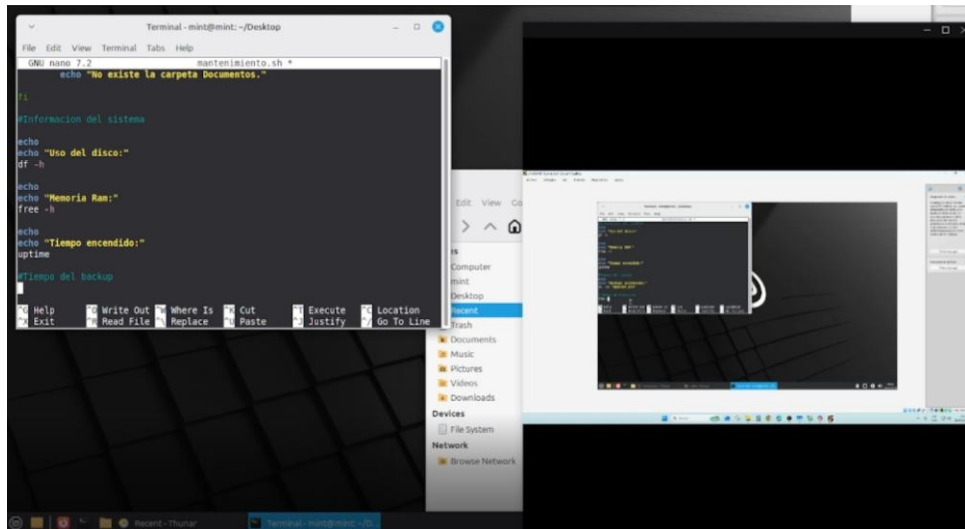
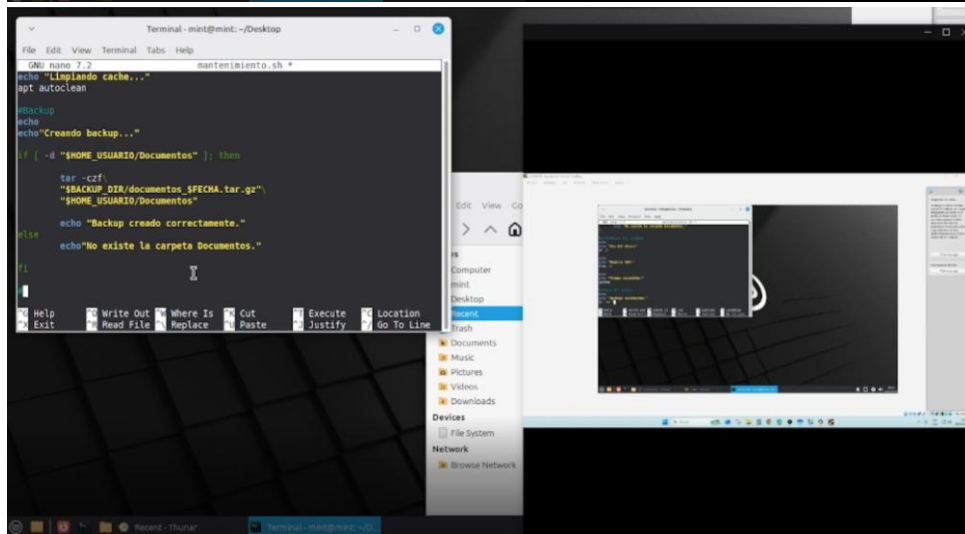
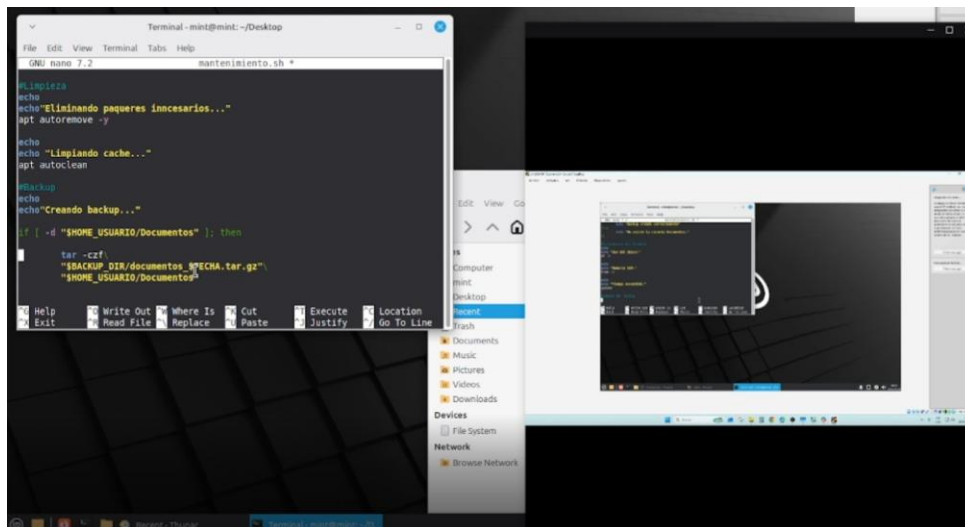
Modos de red en virtual:

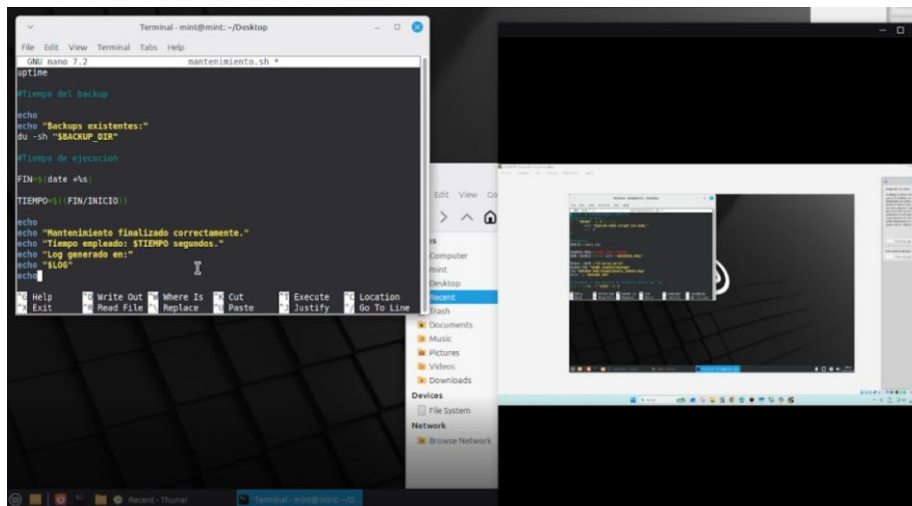
<https://docs.oracle.com/en/virtualization/virtualbox/7.2/user/Introduction.html>)

8. Anexos

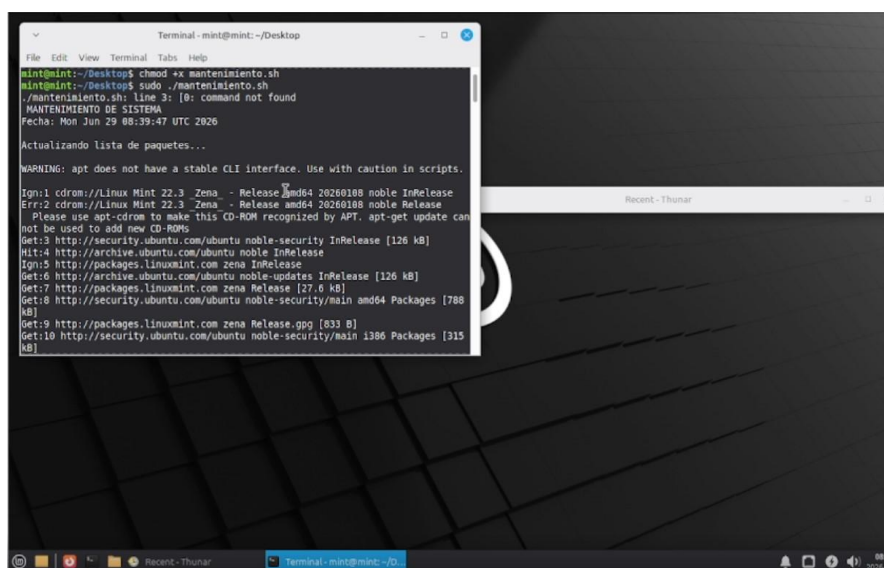
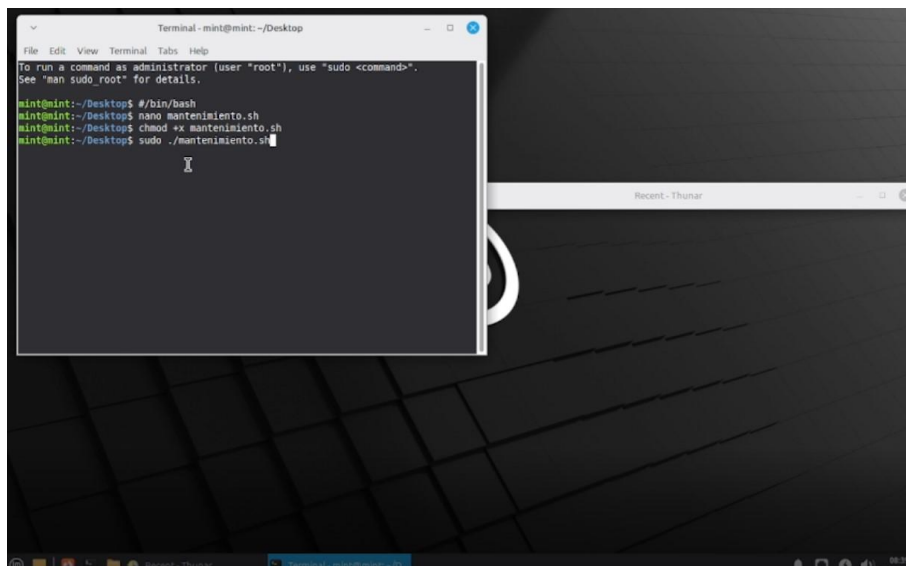
8.1 Capturas de pantalla de la creación de la configuración de la primera máquina virtual







8.3 Capturas de pantalla de la Ejecución del primer script




```

Terminal - mint@mint: ~/Desktop
File Edit View Terminal Tabs Help
get:69 http://archive.ubuntu.com/ubuntu noble-backports/multiverse translation-e
n [340 B]
get:70 http://archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Compone
nts [212 B]
Reading package lists...
E: The repository 'cdrom://Linux Mint 22.3 _Zena_ - Release amd64 20260108 noble
Release' does not have a Release file.

Actualizando programas...

WARNING: apt does not have a stable CLI interface. Use with caution in scripts.

Reading package lists...
Building dependency tree...
Reading state information...
Calculating upgrade...
The following NEW packages will be installed:
  libfwupd3 libtraceevent1 linux-headers-6.17.0-35-generic
  linux-hwe-6.17-headers-6.17.0-35 linux-hwe-6.17-tools-6.17.0-35
  linux-image-6.17.0-35-generic linux-modules-6.17.0-35-generic
  linux-modules-extra-6.17.0-35-generic linux-tools-6.17.0-35-generic
  linux-tools-common vainfo vdpauinfo
The following packages will be upgraded:
 alsa-ucm-conf amd64-microcode apparmor avahi-autoipd avahi-daemon

```

```

Terminal - mint@mint: ~/Desktop
File Edit View Terminal Tabs Help
Eliminando paquetes innecesarios...

WARNING: apt does not have a stable CLI interface. Use with caution in scripts.

Reading package lists...
Building dependency tree...
Reading state information...
0 upgraded, 0 newly installed, 0 to remove and 447 not upgraded.

Limpiando cache...
Reading package lists...
Building dependency tree...
Reading state information...

Creando backup...
No existe la carpeta Documentos.

Uso del disco:

```

Filesystem	Size	Used	Avail	Use%	Mounted on
tmpfs	197M	1.2M	196M	1%	/run
/dev/sr0	2.9G	2.9G	0	100%	/cdrom
/cow	985M	266M	719M	28%	/
tmpfs	985M	0	985M	0%	/dev/shm
tmpfs	5.0M	8.0K	5.0M	1%	/run/lock

```

Terminal - mint@mint: ~/Desktop
File Edit View Terminal Tabs Help

Uso del disco:

```

Filesystem	Size	Used	Avail	Use%	Mounted on
tmpfs	197M	1.2M	196M	1%	/run
/dev/sr0	2.9G	2.9G	0	100%	/cdrom
/cow	985M	266M	719M	28%	/
tmpfs	985M	0	985M	0%	/dev/shm
tmpfs	5.0M	8.0K	5.0M	1%	/run/lock
tmpfs	985M	8.0K	985M	1%	/tmp
tmpfs	197M	152K	197M	1%	/run/user/1000

```

Memoria Ram:

```

	total	used	free	shared	buff/cache	available
Mem:	1.9Gi	982Mi	93Mi	269Mi	1.3Gi	985Mi
Swap:	0B	0B	0B			

```

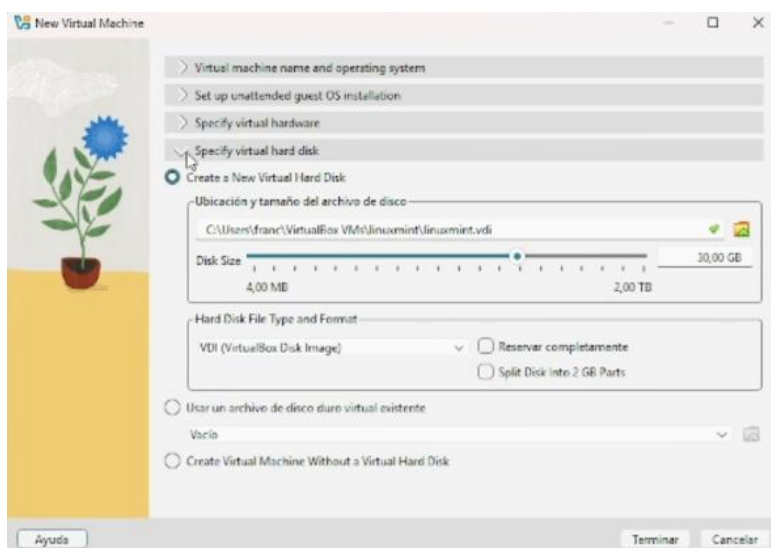
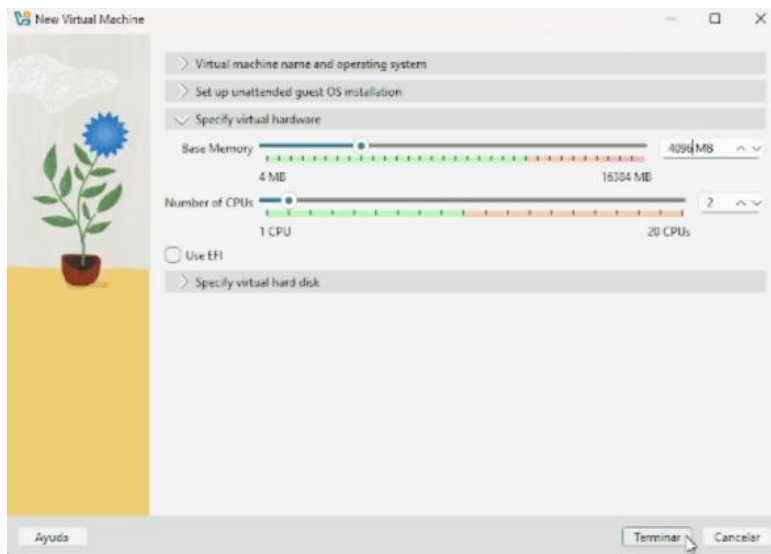
Tiempo encendido:
08:40:06 up 1:09, 1 user, load average: 0.30, 0.08, 0.06

Backups existentes:
16K /home/mint/backups

Mantenimiento finalizado correctamente.
Tiempo empleado: 1 segundos.

```

8.4 Capturas de pantalla creación máquina virtual apta para ejecutar nuestra script



8.5 Capturas de pantalla ejecución del script final en la nueva máquina virtual

```
Terminal - mint@mint: ~/Desktop
File Edit View Terminal Tabs Help
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

mint@mint:~/Desktop$ nano mantenimiento.sh
mint@mint:~/Desktop$ chmod +x mantenimiento.sh
mint@mint:~/Desktop$ sudo ./mantenimiento.sh
MANTENIMIENTO DEL SISTEMA
Fecha: Tue Jun 30 07:45:24 UTC 2026

Limpieza previa agresiva...
Reading Package Lists...
Building dependency tree...
Reading state information...
Removiendo fuentes de repositorios problemáticas...
Actualizando lista de repositorios...

WARNING: apt does not have a stable CLI interface. Use with caution in scripts.

Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Ign:3 http://packages.linuxmint.com zena InRelease
Get:4 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:5 http://packages.linuxmint.com zena Release [27.6 kB]
```

```
Terminal - mint@mint: ~/Desktop
File Edit View Terminal Tabs Help
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

Get:61 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 c-n-f Met
adata [1388 B]
Get:62 http://archive.ubuntu.com/ubuntu noble-backports/multiverse 1386 Packages
[748 B]
Get:63 http://archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Package
s [748 B]
Get:64 http://archive.ubuntu.com/ubuntu noble-backports/multiverse Translation-e
n [348 B]
Get:65 http://archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Compone
nts [212 B]
Get:66 http://packages.linuxmint.com zena/upstream amd64 Packages [50.4 kB]
Get:67 http://packages.linuxmint.com zena/backport 1386 Packages [6480 B]
Get:68 http://packages.linuxmint.com zena/backport amd64 Packages [27.8 kB]
Fetched 12.8 MB in 4s (2894 kB/s)
Reading package lists...
Building dependency tree...
Reading state information...
448 packages can be upgraded. Run 'apt list --upgradable' to see them.

Actualizando programas...

WARNING: apt does not have a stable CLI interface. Use with caution in scripts.
```

```
Terminal - mint@mint: ~/Desktop
File Edit View Terminal Tabs Help
dpkg: error processing archive /var/cache/apt/archives/perl-base_5.38.2-3.2ubuntu3 amd64.deb (--unpack):
dpkg-deb --control subprocess returned error exit status 2tee: /home/mint/backups/mantenimiento 2026-06-30 07-45-24.log: No space left on device
dpkg: error: failed to write status database stanza about 'blueman' to '/var/lib/dpkg/status': No space left on device
E: Sub-process /usr/bin/dpkg returned an error code (2)
Upgrade inicial fallido, intentando limpieza adicional...
WARNING: apt does not have a stable CLI interface. Use with caution in scripts.
E: dpkg was interrupted, you must manually run 'sudo dpkg --configure -a' to correct the problem.
Eliminando paquetes innecesarios...
WARNING: apt does not have a stable CLI interface. Use with caution in scripts.
E: dpkg was interrupted, you must manually run 'sudo dpkg --configure -a' to correct the problem.
Limpiando cache...
Reading package lists...
```

```
Terminal - mint@mint: ~/Desktop
File Edit View Terminal Tabs Help
Limpiando cache...
Reading package lists...
Building dependency tree...
Reading state information...
Creando Backup...
Respaldo de /home/mint/Desktop...
Backup de Desktop creado correctamente.
Respaldo de /home/mint/.config...
Backup de .config creado correctamente.
INFORMACIÓN DEL SISTEMA
Uso del disco:
Filesystem      Size  Used Avail Use% Mounted on
tmpfs            392M  1.2M  391M   1% /run
/dev/sr0         2.9G  2.9G   0 100% /cdrom
/cow             2.0G  457M  1.5G  24% /
tmpfs            2.0G   0 2.0G   0% /dev/shm
tmpfs            5.0M  8.0K  5.0M   1% /run/lock
tmpfs            2.0G  28K  2.0G   1% /tmp
tmpfs            392M 152K  392M   1% /run/user/1000
```

```
Terminal - mint@mint: ~/Desktop
File Edit View Terminal Tabs Help
Backup de desktop creado correctamente.
Respaldo de /home/mint/.config...
Backup de .config creado correctamente.
INFORMACIÓN DEL SISTEMA
Uso del disco:
Filesystem      Size  Used Avail Use% Mounted on
tmpfs            392M  1.2M  391M   1% /run
/dev/sr0         2.9G  2.9G   0 100% /cdrom
/cow             2.0G  457M  1.5G  24% /
tmpfs            2.0G   0 2.0G   0% /dev/shm
tmpfs            5.0M  8.0K  5.0M   1% /run/lock
tmpfs            2.0G  28K  2.0G   1% /tmp
tmpfs            392M 152K  392M   1% /run/user/1000
Memoria RAM:
              total        used         free      shared  buff/cache   available
Mem:    3.861      1.901      1.501      490M      1.261      1.901
Swap:    0B          0B          0B
Tiempo encendido:
07:48:25 up 5 min,  1 user,  load average: 1.16, 0.91, 0.49
```

